



Network-Controlled Custom CMD Panel

Using TCP Sockets

SUBMITTED BY

Hamza Ali

124-BSE-3B-FALL24

Akif Naveed

129-BSE-3B-FALL24

SUBMITTED TO

Dr. Mushtaq Khan

PROJECT TYPE

Semester Project

COURSE

Computer Networks

INSTITUTE

COMSATS University Islamabad, Wah Campus

PROJECT INTRODUCTION



Custom Command Line Panel

Implements a specialized interface for executing network commands efficiently.



Python TCP Sockets

Built using core networking libraries to handle reliable TCP/IP connections.



Client-Server Architecture

Robust separation of concerns where multiple clients can interact with a central server.



Graphical User Interface

A GUI client (Tkinter) acts as the control panel, sending commands remotely.



Predefined Execution

Server executes specific custom commands, ensuring security and predictability.



Educational Purpose

Designed to demonstrate networking protocols and socket programming concepts.

🔒 TCP/IP

```
server_main.py - 80x24

user@host:~$ python3 server.py

[INFO] Server started on 192.168.1.10:5000
[INFO] Waiting for incoming connections...

# New connection detected
[CONN] Client connected: ('192.168.1.15', 54321)

user@client:~$ CMD_EXEC "TIME"

> Processing command: TIME
> Response sent: "2024-12-18 14:30:05"

user@host:~$
```

PROJECT OBJECTIVES

● SYSTEM_GOALS_INIT



01

Real-time TCP Communication

Demonstrate reliable, full-duplex communication between two distinct machines over a local network using Python's socket library.



02

Custom Protocol Design

Design and implement a structured command-line protocol to handle request types, payload formatting, and response parsing efficiently.



03

GUI Client Implementation

Develop a user-friendly graphical interface using Tkinter that simulates a modern terminal environment for sending remote commands.



04

System Simulation

Simulate essential system utilities, network diagnostics, and utility commands to mimic a fully functional remote administration tool.



05

Architecture Mastery

Strengthen theoretical understanding by practically applying the Client-Server model, handling concurrency, and managing socket states.

SYSTEM ARCHITECTURE



TECHNOLOGIES & TOOLS

STACK_CONFIGURATION // MODULES_LOADED

CORE LOGIC



Python 3.x

Primary development language



Threading

Concurrent client handling



NETWORKING



TCP Sockets

Full-duplex communication



JSON

Data serialization protocol



INTERFACE



Tkinter

Native GUI framework



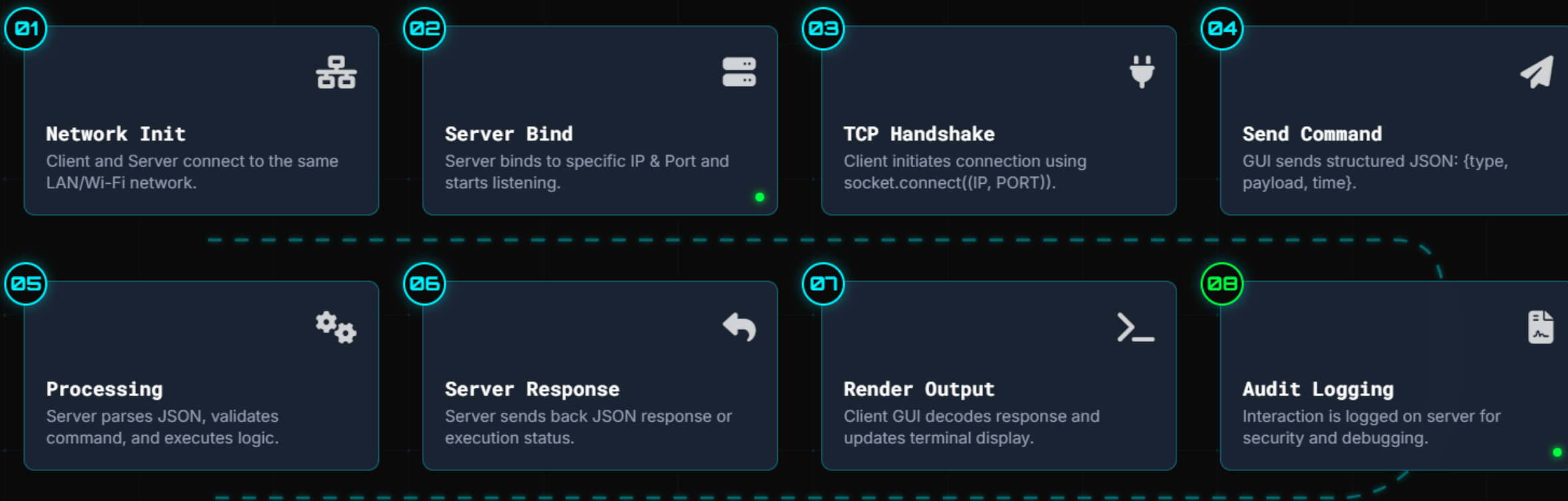
Psutil

System monitoring library



COMMUNICATION WORKFLOW

• ACTIVE_SESSION



COMMANDS & FEATURES

> EXECUTABLE COMMANDS

PROTOCOL: JSON/TCP

> TIME

Retrieves the current server system date and time.

> PING

Checks connection latency and server availability.

> ECHO

Returns the input text sent by the client.

> JOKE

Fetches a random programming joke from the library.

> QUOTE

Displays a motivational quote for the user.

> RAND_FACT

Provides an interesting fact about networking.

> HELP

Lists all available commands and their syntax.

> SYS_LOGS

Admin only: Views the recent 5 server command logs.

STATUS: LISTENING _

SYSTEM CAPABILITIES

VER 1.0.4



Custom GUI Interface

Modern Tkinter-based dashboard simulating a professional command center.



Real-time TCP/IP

Full-duplex communication with low-latency socket handling.



Multi-Client Support

Threaded server architecture handles concurrent connections effortlessly.



Command Logging

Automatic audit trails for every command executed by any client.



Controlled Execution

Restricted command set prevents unauthorized system access.

PROJECT SUMMARY

- ✓ Demonstrates core TCP networking concepts & socket programming.
- ✓ Provides a safe, controlled environment for command execution.
- ✓ Hands-on experience with Client-Server architecture.
- ✓ Integrates Python backend logic with modern Tkinter GUI.

FUTURE EXTENSIONS



User Auth & TLS



File Transfer



Live Monitoring



Shell Access

THANK YOU

SESSION TERMINATED